Smart Parking IoT — Project Report

Oct 13, 2026 · @aZIZOS

1. Subject and objectives

We build Sujet 2, "Smart Parking intelligent avec prédiction de disponibilité": a connected parking that knows in real time which spots are free, predicts the occupancy rate, and helps the driver pick (and reserve) a spot.

The subject imposes four layers plus a DevOps part. Everything below maps one-to-one to those requirements, so nothing the professor asks for is missing:

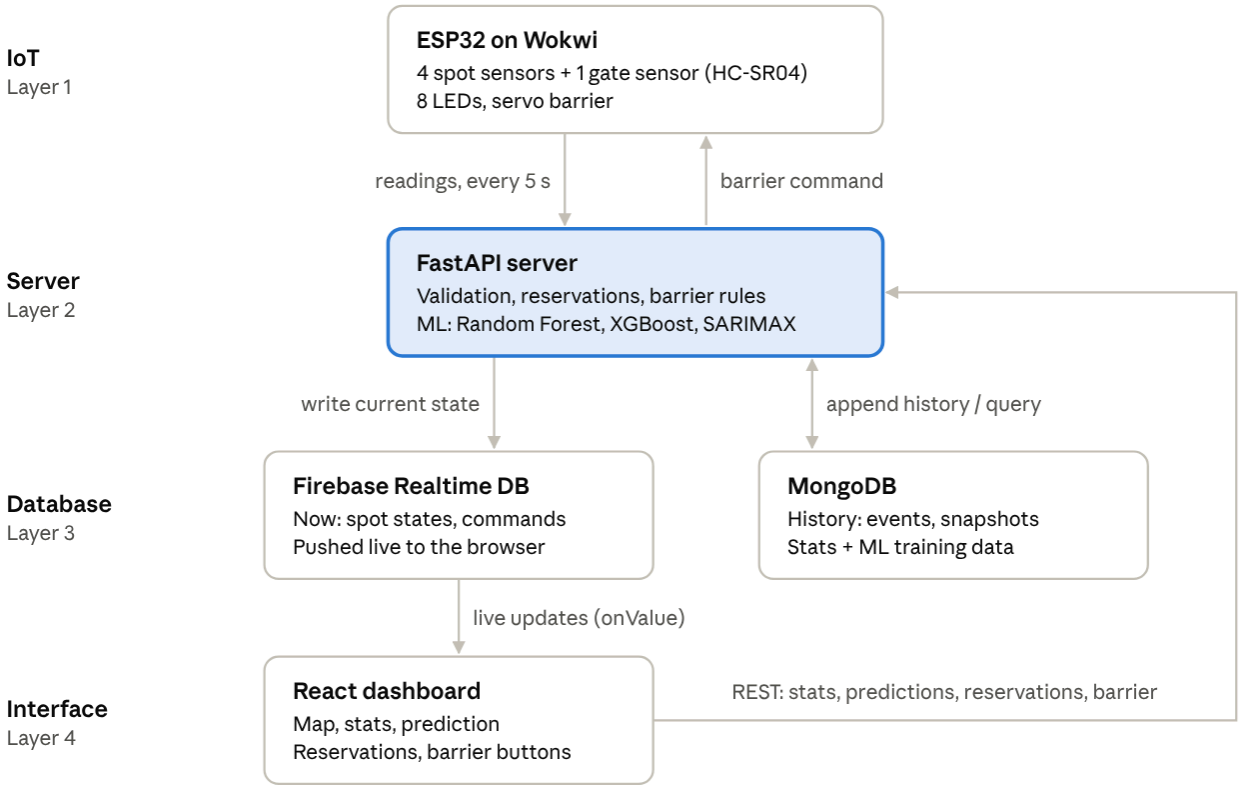
Layer	What the subject requires	What we deliver
IoT (Wokwi)	ESP32, several HC-SR04 as spots, LEDs free/occupied, servo as barrier	ESP32 + 4 spot sensors + 1 gate sensor, 8 LEDs, 1 servo, non-blocking firmware
Server (Python local)	FastAPI receives occupancy; AI predicts occupancy rate from history; compare Random Forest, XGBoost, temporal regression	FastAPI REST API + ML module comparing 4 models (baseline included)
Database	Firebase = instant spot state + barrier commands; MongoDB = occupancy history	Firebase Realtime Database + MongoDB (time-series collection)
Interface (React/HTML/CSS)	Interactive parking map, free/occupied spots, statistics, availability prediction, simulated reservation	React dashboard with 4 pages
DevOps	Docker Compose, GitHub, PyTest, ESP32/API integration tests, performance tests	Compose stack, GitHub Actions CI, PyTest suite, ESP32 simulator, Locust load test

Success criteria for the demo: moving a sensor slider in Wokwi turns a spot red on the dashboard within about 2 seconds; clicking "Open barrier" on the dashboard lifts the servo in Wokwi; the prediction page shows the expected occupancy for the next hours and the model comparison table.

2. Global architecture

FastAPI sits in the middle of the system: the ESP32 and the dashboard talk only to it, and it is the only component that writes to Firebase and MongoDB.

Every write goes through FastAPI; the browser reads live state from Firebase



system architecture · 4 layers, 5 components

One cycle: a car parks → the ESP32 sees the distance drop, turns the LED red and posts the reading → FastAPI validates it, overwrites the spot in Firebase and appends an event to MongoDB → the dashboard, subscribed to Firebase, repaints the spot within about a second. Commands go the other way: a dashboard click becomes a Firebase command, which the API returns to the ESP32 in its next response.

3. IoT layer (Wokwi)

One ESP32 DevKit V1 simulates a 4-spot parking with an entry gate. Four spots keep the wiring readable and the pins fit without using ESP32 boot (strapping) pins.

Components

Component	Qty	Role
ESP32 DevKit V1	1	Reads sensors, drives LEDs and servo, talks to the API over Wi-Fi
HC-SR04 (spots P1–P4)	4	Distance to the car; below the threshold = occupied
HC-SR04 (gate)	1	Detects a car waiting at the entry
Green LED + 220 Ω	4	Spot free
Red LED + 220 Ω	4	Spot occupied
Servo SG90	1	Barrier: 0° closed, 90° open

Proposed pin map

ECHO pins go on GPIO 34/35/36/39, which are input-only and therefore perfect for echo signals.

Spot	TRIG	ECHO	Green LED	Red LED
P1	4	34	16	17
P2	5	35	18	19
P3	23	36 (VP)	25	26
P4	27	39 (VN)	32	33
Gate	13	14	–	–
Servo	21 (signal)	–	–	–

Firmware logic (Arduino C++ with PlatformIO)

1. Connect to Wi-Fi `Wokwi-GUEST` (no password).
2. Every 500 ms, measure each sensor one after another (never two at once, to avoid echo crosstalk on real hardware).
3. A spot is occupied when distance < 50 cm. **Debounce:** the state changes only after 3 identical readings in a row, so a slider passing by doesn't flicker.
4. Update the spot's LEDs immediately (local feedback, works even if the network is down).

5. Send `POST /api/v1/sensors/reading` with the state of all spots: immediately on a change, plus a heartbeat every 5 s.
6. The API's response carries the barrier command (`open` / `close`). Gate logic: car at gate + at least one free spot → open; close again 5 s after the car has passed.
7. Everything is non-blocking (`millis()` timers, no `delay()`), organised as a small state machine for the barrier: `CLOSED → OPENING → OPEN → CLOSING`.

Libraries: `WiFi.h` , `HTTPClient.h` , `ArduinoJson` (v7), `ESP32Servo` .

Example JSON sent by the ESP32:

```
{
  "device_id": "esp32-parking-01",
  "spots": [
    {"spot_id": "P1", "distance_cm": 23.4, "occupied": true},
    {"spot_id": "P2", "distance_cm": 310.0, "occupied": false}
  ],
  "gate_car_detected": false,
  "barrier_state": "closed"
}
```

Reaching the local API from Wokwi (important)

The Wokwi web simulator's public gateway reaches the internet but not our laptop, and the private gateway is only for paying users (Wokwi docs). Options, in order of preference:

1. **Wokwi for VS Code** + PlatformIO: the extension bundles the private IoT gateway, so the ESP32 can reach the API on our machine (Wokwi VS Code config). It also lets us keep the firmware in our Git repo. Licence terms to confirm on first install.
2. **Tunnel** (ngrok or Cloudflare Tunnel): expose `localhost:8000` with a public HTTPS URL and use the free web simulator.
3. **Fallback MQTT:** the ESP32 publishes to a public broker and a small bridge on our side forwards to FastAPI. Only if 1 and 2 fail.

Real hardware note: an HC-SR04 echo is 5 V; on a real ESP32 a voltage divider is needed. Not an issue in Wokwi.

4. Server layer (FastAPI)

The FastAPI server is the single entry point: the ESP32 and the dashboard only talk to it, and it is the only component that writes to Firebase and MongoDB. This keeps the rules (validation, reservations, barrier) in one place and makes everything testable with PyTest.

Stack: Python 3.12, FastAPI, Uvicorn, Pydantic v2 (validation), Motor (async MongoDB driver), firebase-admin, scikit-learn / XGBoost / statsmodels, joblib, python-dotenv / pydantic-settings.

Endpoints (versioned under `/api/v1`)

Method	Route	Used by	Purpose
POST	`/sensors/reading`	ESP32	Receive spot states; returns the barrier command
GET	`/spots`	Dashboard	Current state of every spot
GET	`/stats/occupancy?from&to&step`	Dashboard	Historical occupancy rate (charts)
GET	`/stats/summary`	Dashboard	Peak hours, average occupancy, total entries
GET	`/predictions?horizon=60`	Dashboard	Predicted occupancy rate for the next N minutes
GET	`/models/compare`	Dashboard	MAE / RMSE / R^2 of each model
POST	`/reservations`	Dashboard	Reserve a free spot for 15 min (simulated)
DELETE	`/reservations/{id}`	Dashboard	Cancel a reservation
POST	`/barrier/command`	Dashboard	Manual open / close
GET	`/health`	Docker, CI	Liveness check

Business rules

- **Validation:** Pydantic rejects unknown spot IDs, negative distances, or malformed JSON with a 422.
- **Change detection:** on each reading, the server compares with the last known state; only a real change writes an event to MongoDB (the heartbeat just refreshes `last_seen`).
- **Snapshots:** every minute, a background task stores the global occupancy rate (e.g. 3/4 = 0.75). This is the series the AI learns from.
- **Reservations:** a reserved spot is shown orange; it expires automatically after 15 min; a reserved spot cannot be reserved twice (409 Conflict).

- **Barrier:** opens automatically if a car is at the gate and free spots > 0; a manual command from the dashboard overrides it.
- **Offline detection:** no heartbeat for 15 s → the device is marked offline on the dashboard.
- **Security (simple but real):** the ESP32 sends an `X-API-Key` header; CORS limited to the dashboard origin; secrets in `.env`, never in Git.

Code organisation

Layered, so each part is small and testable: `routers/` (HTTP only) → `services/` (business rules) → `repositories/` (MongoDB, Firebase access) + `schemas/` (Pydantic models) + `ml/` (training and prediction).

5. Database layer

The split follows the subject: Firebase holds "now" (small, overwritten, pushed live to the browser), MongoDB holds "history" (append-only, queried for stats and AI training).

	Firebase Realtime Database	MongoDB 7
Holds	Current state of each spot, barrier state and commands, active reservations	Every state change, minute snapshots, reservations log, barrier log, predictions
Write pattern	Overwrite the same keys	Append new documents
Read pattern	Dashboard subscribes with `onValue` → instant updates, no polling	Aggregations (per hour, per weekday), ML dataset export
Runs	Firebase cloud, free Spark plan (or the Firebase Emulator in Docker for offline dev)	Docker container

Firebase structure (JSON tree)

```
{
  "parking": {
    "spots": {
      "P1": {"status": "occupied", "distance_cm": 23.4, "updated_at":
1760342400}
    },
    "summary": {"free": 2, "occupied": 1, "reserved": 1, "total": 4},
    "device": {"online": true, "last_seen": 1760342400}
  },
  "barrier": {
    "state": "closed",
    "command": {"action": "open", "source": "dashboard", "issued_at":
1760342400}
  }
}
```

`status` is one of `free`, `occupied`, `reserved`. Security rules: the browser can only read; only the server (service account via firebase-admin) can write.

MongoDB collections

Collection	One document =	Key fields	Index
`occupancy_events`	One spot changing state	`spot_id`, `occupied`, `distance_cm`, `ts`	`(spot_id, ts)`
`occupancy_snapshots` (time-series)	Whole parking, once a minute	`ts`, `occupied`, `total`, `rate`	time-series on `ts`
`reservations`	One reservation	`spot_id`, `user_name`, `created_at`, `expires_at`, `status`	`(spot_id, status)`
`barrier_events`	One open/close	`action`, `source` (auto/dashboard), `ts`	`ts`
`predictions`	One prediction served	`model`, `horizon_min`, `predicted_rate`, `ts`	`ts`

`occupancy_snapshots` uses MongoDB's native time-series collection type, made for exactly this kind of data (compact storage, fast range queries).

6. AI module: predicting occupancy

The model predicts the parking's occupancy rate (0 to 1) at t + 15, 30 and 60 minutes. This is a regression on a time series, and we compare 4 models on the same data with a time-based split.

Data: three sources, combined

Four Wokwi spots running a few days will never give enough history to train a model, so we plan for it from the start:

1. **Real dataset — UCI Parking Birmingham:** 35,717 records from 30 car parks (capacity, occupancy, timestamp), 4 Oct–19 Dec 2016, 8:00–16:30, licence CC BY 4.0. Used to train and evaluate the models on real behaviour.
2. **Synthetic history for our 4-spot parking:** a Python generator creates 90 days at 5-minute steps with realistic patterns (morning and lunch peaks, quieter weekends, random noise, occasional "event" days). Seeded so results are reproducible.
3. **Live Wokwi data** stored in MongoDB: used for the demo and appended to the training set.

Features

Group	Features	Why
Calendar	hour, minute, day of week, is_weekend, is_holiday	Parking follows the clock and the week
Cyclical encoding	sin/cos of hour and of weekday	23h and 0h are close; plain numbers hide that
Lags	rate at t−5, t−15, t−30, t−60 min; same time yesterday; same time last week	Recent past is the strongest predictor
Rolling stats	mean and std over the last 30 and 60 min	Captures the trend (filling up or emptying)

Models compared

Model	Library	Role in the comparison
Naive baseline (same time last week)	–	The bar to beat; any model worse than this is useless

Model	Library	Role in the comparison
Temporal regression	statsmodels SARIMAX (or linear regression on lags)	Classic time-series approach, interpretable
Random Forest Regressor	scikit-learn	Robust non-linear model, little tuning
XGBoost Regressor	xgboost	Gradient boosting, usually the most accurate on tabular data

Evaluation protocol

- **Time-based split, never random:** train on the first 80% of the timeline, test on the last 20%. A random split leaks the future into training and gives fake scores.
- **Cross-validation:** `TimeSeriesSplit` (5 folds) for hyper-parameter tuning.
- **Metrics:** MAE (main metric, easy to explain: "we're off by X points of occupancy on average"), RMSE, R^2, plus training and prediction time.
- **Explainability:** feature importance for RF and XGBoost, shown on the dashboard.
- **Output:** the best model is saved with joblib (`models/best_model.joblib`) with its metadata (date, features, scores); the API loads it at start-up. A `train.py` script re-runs the whole pipeline in one command.

From rate to spots: predicted free spots = total × (1 – predicted rate), rounded. The dashboard says, for example: "In 30 min: about 1 free spot (72% full)".

7. Web interface (React)

A React single-page app with four pages. Live spot colours come straight from Firebase; everything else (stats, predictions, reservations, barrier) goes through the API.

Stack: React 18 + Vite, React Router, Firebase JS SDK (`onValue` listener), Recharts (charts), Axios, Tailwind CSS. The parking map is a hand-drawn SVG: each spot is a rectangle whose colour is bound to its Firebase status.

Page	What the user sees and does
Live map	SVG plan of the parking: green = free, red = occupied, orange = reserved, grey = sensor offline. Counter "2 / 4 free". Barrier status + Open / Close buttons. Click a free spot → reserve it

Page	What the user sees and does
Statistics	Occupancy over the day (line), average by hour of the week (heatmap), peak hours, number of entries, average parking duration
Prediction	Forecast curve for the next hours with the current rate, "best time to come" hint, model comparison table (MAE, RMSE, R^2) and feature importance
Reservations	Simulated booking form (name, spot, duration), list of active reservations with countdown, cancel button

UX details that make the demo convincing: a spot changes colour with a short animation; a toast appears when the barrier opens; the device's online/offline badge is always visible in the header; the layout works on a phone (the driver's use case).

8. DevOps and testing

One command, `docker compose up`, must start the whole local stack; one push to GitHub must run all tests automatically.

Docker Compose services

Service	Image / build	Port	Notes
`api`	`./backend` (python:3.12-slim)	8000	FastAPI + Uvicorn, healthcheck on `/health`
`mongo`	`mongo:7`	27017	Named volume so history survives restarts
`mongo-express`	`mongo-express`	8081	Optional: browse the data during development
`frontend`	`./frontend` (Node build → nginx)	5173 / 80	Serves the React build
`firebase-emulator`	optional profile	9000	Work offline without the real Firebase project
`trainer`	`./backend`, run once	–	`docker compose run trainer` retrains the models

Secrets (Firebase service account, API key) live in `.env` and a mounted JSON file, both in `.gitignore`. A committed `.env.example` documents every variable.

GitHub workflow

- Branches: `main` (always working) ← `develop` ← feature branches (`feat/iot-firmware`, `feat/api-sensors` …), merged by pull request.
- Commits follow Conventional Commits (`feat:`, `fix:`, `test:`, `docs:`).
- **GitHub Actions CI** on every push and PR: lint (ruff + ESLint) → PyTest with a MongoDB service container → coverage report → frontend build → Docker images build. Optional: compile the firmware with PlatformIO.

Test pyramid

Level	Tool	What is tested	Examples
Unit	PyTest	Pure logic, no network	debounce, rate computation, reservation expiry, feature engineering, Pydantic validation
API	PyTest + FastAPI `TestClient` / httpx	Each endpoint, status codes, errors	422 on bad JSON, 401 without API key, 409 on double reservation
Integration ESP32/API	PyTest + `esp32_simulator.py`	The exact JSON contract the firmware sends, end to end through MongoDB and Firebase	reading → event stored → Firebase updated → barrier command returned
ML	PyTest	Pipeline sanity	no future leakage in features, model beats the naive baseline, predictions within [0, 1]
Performance	Locust	API under load	200 simulated sensors posting every second; target p95 latency < 200 ms, 0% errors

The **ESP32 simulator** is a small Python script that sends the same JSON as the firmware (shared JSON schema in `docs/contracts/`). It lets us test and demo the backend without opening Wokwi, and it is what Locust reuses for the load test.

Coverage target: ⩾ 80% on the backend.

9. Tech stack at a glance

Layer	Technology	Why this one
Simulation	Wokwi for VS Code + PlatformIO	Firmware lives in the repo, local network access to the API
Firmware	Arduino C++ on ESP32, ArduinoJson 7, ESP32Servo	Standard, well documented, matches the course (ESP32)
API	Python 3.12, FastAPI, Uvicorn, Pydantic v2	Imposed by the subject; async, auto docs at `/docs`
Real-time DB	Firebase Realtime Database (firebase-admin / JS SDK)	Imposed; pushes changes to the browser instantly
History DB	MongoDB 7 (time-series collection), Motor	Imposed; flexible documents, aggregation pipeline
AI	pandas, scikit-learn, XGBoost, statsmodels, joblib	Covers the 3 model families the subject names
Front-end	React 18, Vite, Recharts, Tailwind CSS, React Router	Fast dev server, simple charts, responsive layout
Containers	Docker, Docker Compose	Imposed; one-command start
CI/CD	GitHub + GitHub Actions	Imposed (GitHub); free for student repos
Tests	PyTest, pytest-cov, httpx, Locust	Unit, API, integration and performance tests
Quality	ruff (Python lint/format), ESLint + Prettier	Clean, consistent code across the team

10. Repository, roadmap and risks

Repository structure (monorepo)

```
smart-parking/
├── README.md              # step-by-step build log + code explanations
├── docker-compose.yml
├── .env.example
├── .github/workflows/ci.yml
├── firmware/              # ESP32 (PlatformIO + Wokwi)
│   ├── platformio.ini
│   ├── wokwi.toml
│   ├── diagram.json       # Wokwi circuit
│   └── src/main.cpp
├── backend/               # FastAPI + ML
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── routers/  services/  repositories/  schemas/
│   │   └── ml/            # features.py, train.py, predict.py
│   ├── data/              # generator + downloaded dataset
│   ├── models/            # saved .joblib models
│   ├── tests/             # unit, api, integration, ml
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/              # React + Vite
│   ├── src/pages/  src/components/  src/services/
│   └── Dockerfile
├── tools/
│   ├── esp32_simulator.py
│   └── locustfile.py
└── docs/                  # architecture, API contract, final report
```

Roadmap (build order)

Each phase ends with something that runs and is demoable; the README gets a new step after each one.

1. **Setup:** GitHub repo, folder structure, Docker Compose with MongoDB, empty FastAPI with `/health`, CI skeleton, Firebase project.
2. **IoT:** Wokwi circuit (`diagram.json`), firmware reading the 5 sensors, LEDs, debounce, servo state machine. Tested alone first (serial monitor).
3. **API + databases:** `/sensors/reading`, writes to MongoDB and Firebase, barrier command loop, ESP32 simulator, first PyTest suite.

4. **Wokwi ↔ API link:** connect the simulated ESP32 to the local API (VS Code gateway or tunnel). End-to-end demo of a spot turning red.
5. **Dashboard:** live map from Firebase, barrier buttons, reservations, statistics page.
6. **AI:** dataset (UCI + generator), features, 4 models, comparison, best model served by `/predictions`, prediction page.
7. **Quality:** integration and ML tests, Locust performance test, coverage ⩾ 80%, full CI, Docker images.
8. **Delivery:** final report, architecture diagram, demo script and video, slides.

Risks and fallbacks

Risk	Impact	Fallback
Wokwi can't reach the local API	No live IoT demo	Tunnel (ngrok / Cloudflare), else ESP32 simulator for the demo
Too little real history to train on	Weak predictions	UCI dataset + synthetic generator from day one
Firebase credentials leaked to GitHub	Security issue, project compromised	`.gitignore` + `.env.example` + secret scanning; rotate the key if it happens
ML scores look "too good"	Wrong conclusions in the report	Strict time-based split, naive baseline, leakage test in PyTest
Scope too large for the deadline	Unfinished project	Phases 1–5 are the core; AI extras (SARIMAX tuning, Firebase emulator) are optional